Evaluating Vision-Language Models for Automated PCB Quality Control on full-board images

Alberto Robazza

July 2025

Abstract

This project investigates the application of image captioning models to automate quality control processes in the context of printed circuit boards (PCBs). Two state-of-the-art vision-language models—GIT from Microsoft and OpenCLIP—were trained and evaluated on a custom dataset consisting of annotated PCB images paired with textual descriptions of defects. The objective was to determine whether such models could identify and describe manufacturing defects purely from visual input. Despite successful training and convergence in both experiments, results show that the models fail to detect visual differences associated with defects, often producing identical or incorrect captions regardless of input variations. These findings suggest current image captioning approaches may not yet be suitable for fine-grained anomaly detection tasks in visually homogeneous domains like PCBs. Future work may explore more targeted visual input (e.g., cropped defect regions) and larger, more diverse training datasets to improve sensitivity and performance in such applications.

1 Introduction

Printed Circuit Boards (PCBs) serve as the foundational platform for many modern electronic systems such as power electronic converters[6], sensors[26] and computers[16]. They provide both the mechanical support and the electrical interconnection among various electronic components. Their critical role in enabling the functionality and reliability of electronic devices makes the integrity of PCB manufacturing processes an essential focus in electronics engineering and quality control.

Minor defects introduced during PCB fabrication or assembly—such as the omission of components, the formation of unintended solder bridges, misalignment of parts or the ones in [22]—can significantly compromise device performance. These flaws not only result in malfunctioning products but can also

lead to systemic failures that necessitate costly recalls, damage brand reputation, and disrupt supply chains. Therefore, ensuring high manufacturing fidelity and robust inspection protocols is vital for maintaining product quality and mitigating economic risks. For this reason, some automatic inspections techniques have been already tested in the past[18].

1.1 Motivation

The primary objective of this study is to evaluate the performance of image captioning systems when applied to various images of Printed Circuit Boards (PCBs). This task is particularly relevant due to the complexity and variability inherent in PCB layouts, which pose unique challenges for visual recognition and descriptive modeling. To explore different methodological approaches and potentially enhance overall performance, two distinct types of models were employed. Each model represents a separate strategy for addressing the captioning task, allowing for a comparative analysis of their effectiveness across multiple PCB image datasets.

1.2 Problem description

The model is required to accurately detect and identify a range of defects present on Printed Circuit Boards (PCBs) from input images. These defects may include, but are not limited to, missing components, soldering issues, misalignments, and surface contamination. The overarching goal of this work is to select an appropriate model architecture, apply fine-tuning techniques to adapt it to the specific characteristics of PCB imagery, and rigorously evaluate its performance using established metrics. Based on the experimental results, a comprehensive analysis will be conducted to assess the model's effectiveness, draw meaningful conclusions, and identify potential directions for future improvement.

2 Literature review

2.1 Kind of models

A recent comparison [23] divides the models according to their capabilities: *(i)* older captioning models are capable of performing captioning tasks alone, such as BLIP-2 [17] or VinVL [35], and *(ii)* more recent general purpose models, such as Gemma 3 [28], Llama 3 [11], InstructBLIP [5], Microsoft GIT [31]. Along with other kinds of tasks, these recent models are also capable of performing image captioning and still obtaining good performance compared to specialized captioning models.

2.2 Transformer

Transformer[30] is a model architecture fully based on an attention mechanism to draw global dependencies between input and output.

Unlike traditional neural sequence transduction models that utilize an encoder-decoder architecture, Transformers operate by first mapping an input sequence of symbol representations $(x_1, \ldots, x_n)$ to a sequence of continuous representations $z = (z_1, \ldots, z_n)$ via the encoder. The decoder then generates an output sequence of symbols $(y_1, \ldots, y_m)$ one element at a time. At each decoding step, the model operates auto-regressively, using the previously generated symbols as additional input for predicting the next symbol.

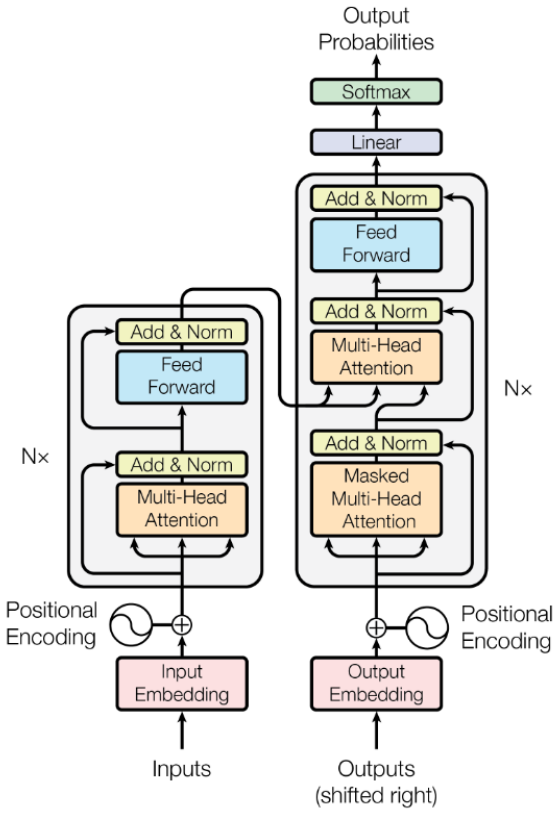


Figure 1: Transformer architecture

The Transformer architecture consists of stacked self-attention layers and point-wise fully connected layers in both the encoder and decoder components, as illustrated in the left and right halves of Figure 1, respectively.

2.3 GIT

The Generative Image-to-Text Transformer (GIT)[31], introduced by Microsoft in 2022, represents a unified framework for vision-language tasks such as image captioning and visual question answering. Unlike earlier generative models, which often rely on heterogeneous architectures with modality-specific encoders and decoders or require auxiliary components for alignment between vision and language representations, GIT adopts a streamlined and end-to-end design. Specifically, it consists of a single vision encoder to process the input image and a text decoder to generate the corresponding natural language output. This simplification not only reduces architectural complexity but also maintained a competitive performance across a range of multimodal benchmarks.

2.4 Architecture

The image encoder is derived from a contrastively pre-trained model [34]. It receives raw images as input and outputs a compact two-dimensional feature map. This feature map is then flattened into a sequence of feature vectors. These vectors are projected into a shared latent space of dimension D using a linear projection layer followed by layer normalization. The resulting representations are used as input to the text decoder.

The text decoder is implemented as a Transformer module composed of multiple blocks, each containing a self-attention layer followed by a feed-forward network. Input text is first tokenized and embedded into D-dimensional vectors. Positional encodings are added, and the embeddings are normalized with layer normalization. The sequence of image features is concatenated with the text embeddings and provided as input to the Transformer decoder. The seq2seq attention mask is employed to control the flow of information during decoding. In this configuration, each text token is allowed to attend to all preceding text tokens as well as all image tokens. Conversely, the image tokens are permitted to attend to one another without restriction. This design contrasts with a strictly unidirectional attention mask, in which image tokens would not have full visibility of other image tokens, thus limiting their contextual interactions.

Text decoding is performed in an auto-regressive manner. The generation begins with a special [BOS] token and proceeds sequentially until the [EOS] token is predicted or a maximum sequence length is reached. Notably, the text decoder is randomly initialized and trained from scratch.

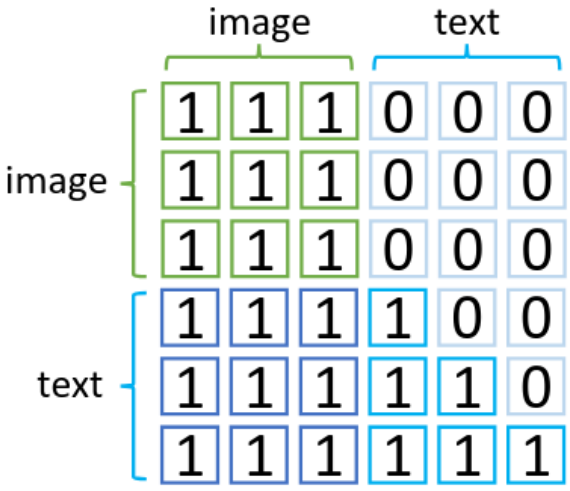


Figure 2: seq2seq attention mask applied to the transformer. Only if the value at position (i,j) is 1, then input i can depend on input j.

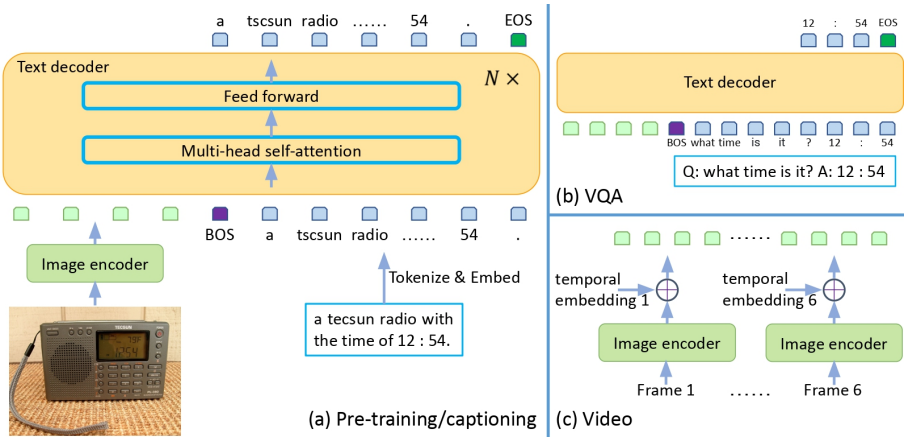


Figure 3: Schema representing GIT architecture.

This study will primarily focus on Sections (a) and (b) of the referenced image. Section (a) outlines the pre-training strategy employed for the model, detailing how visual and textual representations are jointly learned to enable effective multimodal understanding. Section (b), on the other hand, illustrates the model's application to the Visual Question Answering (VQA) task, wherein it generates contextually relevant answers based on image content and natural language queries. By analyzing these two components, the investigation aims

to assess both the foundational training methodology and the model's capacity for downstream task generalization.

2.4.1 Pretraining

The model is trained using a language modeling (LM) objective, defined by the following loss function: $l = \frac{1}{N+1}\Sigma_{i=1}^{N+1} CE(y_i, p(y_i|I, \{y_j, j = 0, ..., i-1)\})$ where CE denotes the cross-entropy loss with a label smoothing factor of 0.1. An alternative approach is masked language modeling (MLM), which typically predicts 15% of the input tokens per iteration. Consequently, predicting all tokens requires at least $1/0.15 = 6.71/0.15 = 6.7$ epochs. In contrast, the LM approach predicts all tokens in every iteration, making it more efficient for large-scale pre-training. Ablation studies[15] also demonstrate that LM outperforms MLM under limited training epochs. In the absence of image input, the architecture reduces to a decoder-only language model, structurally similar to GPT-3[2].

2.5 Florence 2

Florence 2[33], introduced by Microsoft in 2023, is a unified vision-language foundation model designed to handle a broad spectrum of visual understanding tasks. The model operates using task-specific instructions provided through natural language prompts, enabling it to adapt dynamically to various objectives within a single framework. The outputs are returned in textual format, allowing Florence 2 to perform tasks such as image captioning, object detection and phrase grounding without the need for task-specific architectural modifications. This prompt-based interface facilitates flexible interaction with the model and supports generalization across diverse vision-language tasks, making it a viable choice for scalable multimodal applications.

2.5.1 Architecture

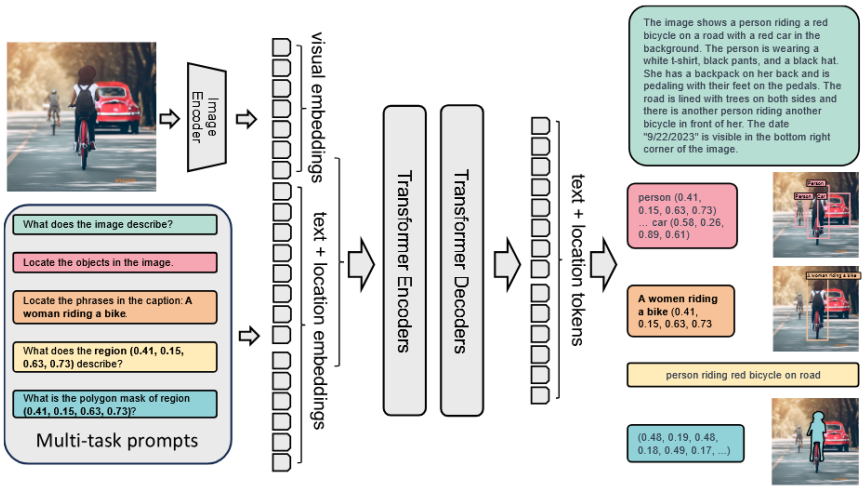


Figure 4: Schema representing Florence 2 architecture.

The input image is initially processed by a DaViT-based vision encoder [9], which transforms the visual content into a sequence of visual token embeddings. These image-derived tokens are subsequently fused with textual embeddings corresponding to the input prompt. The combined multimodal representation is then passed through a unified encoder-decoder architecture, which integrates both visual and linguistic information to generate the appropriate textual response. This design enables the model to handle a wide range of vision-language tasks within a single, end-to-end framework.

2.5.2 Pretraining

Florence 2 is pretrained on the database FLD-5B, developed together with the model. To develop a general-purpose vision foundation model, a range of multitask learning objectives is defined, each designed to target specific aspects of visual understanding. These objectives are structured around the principles of spatial hierarchy and semantic granularity. The proposed multitask framework integrates three distinct learning objectives, each corresponding to a different level of semantic abstraction:

- Image-level understanding tasks aim to capture high-level semantic representations of visual input, enabling the model to interpret global image context and semantic relationships expressed in natural language. Representative tasks include image classification, image captioning, and visual question answering (VQA).
- Region/pixel-level recognition tasks support precise localization and identification of objects or entities within an image. These tasks capture spa-

tial relationships and object interactions and include object detection, segmentation, and referring expression comprehension.

- Fine-grained visual-semantic alignment tasks require detailed correspondence between textual elements and visual regions. These tasks involve identifying specific objects, attributes, or relationships described in the text and aligning them to their corresponding visual representations. This facilitates a nuanced understanding of localized visual semantics and multimodal interactions.

By integrating these objectives within a unified multitask learning framework, the model is trained to handle varying levels of visual granularity and semantic complexity. This approach enables the development of a universal visual representation that extends beyond surface-level recognition to support comprehensive visual understanding.

2.6 CLIP

CLIP (Contrastive Language–Image Pre-training), developed by OpenAI and released in 2021 [19], is a multimodal model trained to learn visual representations from natural language supervision. The model is optimized to predict the correct pairing between an image and its corresponding caption from a set of possible combinations. By jointly training a vision encoder and a text encoder to map images and textual descriptions into a shared embedding space, CLIP enables natural language to serve as interface for referring to visual concepts. This alignment allows the model to achieve good performance on vision-language tasks, such as zero-shot classification, without requiring task-specific fine-tuning.

2.6.1 Architecture

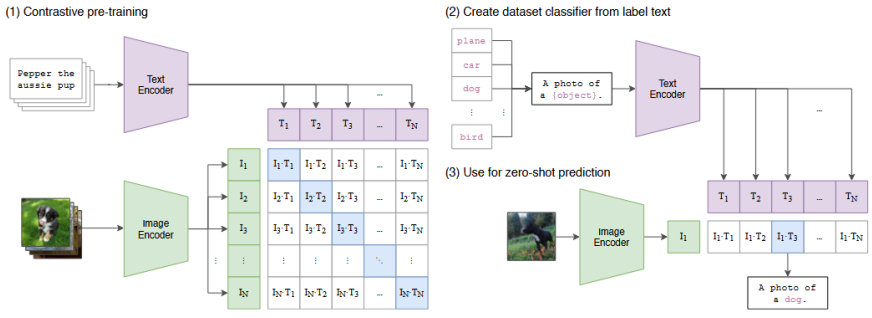


Figure 5: Schema representing CLIP architecture.

CLIP employs a joint training strategy in which an image encoder and a text encoder are optimized simultaneously to predict the correct associations between

images and their corresponding textual descriptions within a batch of training examples. Specifically, the model is trained using a contrastive loss that encourages the alignment of paired (image, text) samples while pushing apart unpaired combinations. CLIP is trained from scratch, with both the image and text encoders not initialized. This end-to-end training approach allows the model to learn a shared embedding space that effectively captures semantic correspondences between visual content and natural language, enabling zero-shot transfer to a wide range of downstream vision-language tasks. CLIP incorporates two distinct architectural choices for its image encoder, each offering different trade-offs in terms of performance and efficiency.

- The first option is a modified version of the ResNet-50 architecture [12], which integrates enhancements from the ResNet-D variant [14]. ResNet-D is a modification to the original ResNet architecture, introduced as an improvement over ResNet-B. While ResNet-B addresses the information loss in path A of the downsampling block by switching the stride between the first two convolutions, ResNet-D extends this idea to path B. In the original design, the 1×1 convolution in path B with a stride of 2 discards three-quarters of the input feature map. To mitigate this, ResNet-D introduces a 2×2 average pooling layer with a stride of 2 placed before the convolution, and changes the convolution's stride to 1. This adjustment helps preserve input information while maintaining the output shape. Empirical results show that this modification performs well in practice with minimal impact on computational cost. Additionally, the model employs anti-aliased rect-2 blur pooling [36], a technique designed to preserve shift-invariance and improve the model's robustness to small image transformations.

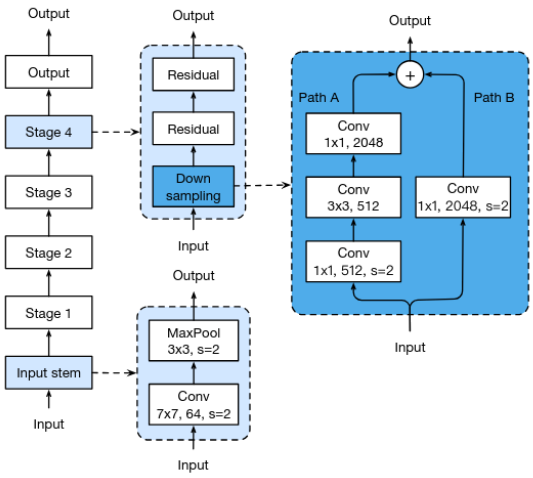


Figure 6: Architecture of ResNet-50

- The second option utilizes the Vision Transformer (ViT) architecture[10]. ViT adapts the Transformer architecture, originally designed for sequential data, to the domain of image recognition. To enable processing of 2D visual input, the input image $x \in \mathbb{R}^{H \times W \times C}$ is divided into a sequence of non-overlapping 2D patches of resolution (P,P)(P,P), resulting in $N = \frac{H \cdot W}{P^2}$ patches. Each patch is flattened and linearly projected into D dimensions using a trainable linear projection. This sequence of patch embeddings is prepended with a learnable classification token, analogous to the token [class] used in BERT[8], whose final representation is used for downstream classification tasks. Positional information is preserved by adding learnable 1D positional embeddings to the input sequence. The resulting vector sequence is passed through a standard Transformer encoder composed of alternating layers of multi-headed self-attention and MLP blocks, each preceded by Layernorm (LN) and followed by residual connections. The two layers contained in MLP feature GELU nonlinearity. Unlike convolutional neural networks (CNNs), ViT introduces significantly less image-specific inductive bias. While CNNs incorporate locality, translation equivariance, and 2D neighborhood structure natively into their layers, ViT only uses such spatial structure during the initial patch extraction and at fine-tuning time for position embedding adjustment. As a result, spatial relationships between patches must be learned during training.

  A hybrid architecture variant of ViT replaces raw image patches with patches extracted from CNN feature maps. These CNN-derived features are similarly projected into the Transformer dimension and fed into the encoder along with position and classification embeddings, preserving the same pipeline structure.

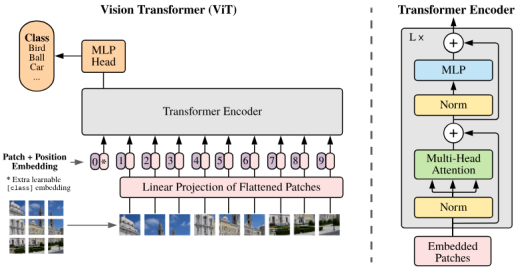


Figure 7: Architecture of ViT

The text encoder in CLIP is based on the Transformer architecture originally proposed by Vaswani et al. However, it incorporates several architectural and training modifications as outlined in Radford et al. [21]. These adaptations are mostly based on the details of the OpenAI GPT model [20] with some

modifications.

Layer normalization[1] is applied at the input of each sub-block, following the approach used in pre-activation residual networks[13]. An additional layer normalization is introduced after the final self-attention block. To account for the accumulation along the residual path as model depth increases, a modified initialization is adopted: the weights of residual layers are scaled at initialization by a factor of $\frac{1}{\sqrt{N}}$, where N represents the number of residual layers. The vocabulary is expanded to 50,257 tokens. The context length is increased from 512 to 1,024 tokens, and a larger batch size of 512 is employed.

2.6.2 Pretraining

A new dataset consisting of 400 million (image, text) pairs was constructed from a wide range of publicly available sources on the Internet. To ensure broad coverage of visual concepts, (image, text) pairs were collected by querying with a set of 500,000 predefined text prompts. To maintain approximate class balance, up to 20,000 pairs were included per query. The final dataset contains a total word count comparable to that of the WebText dataset used for training GPT-2. This dataset is referred to as WebImageText (WIT).

Unlike other apporach (such as VirTex[7]), where an image CNN and a text transformer are trained from scratch, the pre-training method for CLIP has been predicting only which text as a whole is paired with which image and not the exact words of that text. Starting with the same bag-of-words encoding baseline, the model uses a contrastive objective in place of predictive objective, as this approache suppose to lead to superior performance[29] and observed a 4-fold efficiency improvement in the rate of zero-shot transfer to ImageNet.

Given a batch of N (image, text) pairs, CLIP is trained to identify the correct pairings among the $N \times N$ possible (image, text) combinations within the batch. To achieve this, CLIP jointly trains an image encoder and a text encoder to construct a shared multi-modal embedding space. The training objective maximizes the cosine similarity between the embeddings of the NN correct image-text pairs, while minimizing the cosine similarity for the $N^2 - N$ incorrect pairings. This batch-based contrastive training approach follows the multi-class N-pair loss[27], and was recently adapted for contrastive image-text representation learning in medical imaging[37], although in this case some simplifications have been made.

2.7 OpenCLIP

OpenCLIP [4] is an open-source implementation of the CLIP model, designed to provide a reproducible and extensible framework for contrastive vision-language pre-training. Released in 2024, OpenCLIP supports a wide range of model variants, training configurations, and datasets, facilitating both academic research and industrial deployment.

In this study, OpenCLIP serves as the base architecture for fine-tuning on a custom dataset of Printed Circuit Board (PCB) images. The selected con-

figuration utilizes the ViT-B/32 architecture as the image encoder—a Vision Transformer model with a patch size of 32 pixels—and employs the pre-trained checkpoint laion2b_s34b_b79k, which was trained on 2 billion image-text pairs from the LAION-2B dataset. This specific model was chosen for its strong performance across diverse visual tasks and its suitability for adaptation to specialized domains such as PCB inspection.

2.7.1 Pretraining

For model scaling, CLIP architectures are selected using visual encoders ViT-B/32, ViT-B/16, ViT-L/14, ViT-H/14, and ViT-g/14, with the corresponding text encoders scaled proportionally. In terms of data scale, LAION-80M (an 80-million sample subset of LAION-400M), LAION-400M[25], and LAION-2B (another way to refer to LAION-5B[24]) are utilized. Training duration is controlled by selecting scales based on the number of samples seen: 3 billion, 13 billion, and 34 billion.

Compared to the original CLIP training setup, larger batch sizes are employed, with the learning rate adjusted accordingly. For each sample count scale, a separate training run is conducted using a cosine annealing learning rate schedule adapted to the respective number of samples.

Training is performed using mixed precision. Depending on empirical findings, either float16 or bfloat16 formats are applied.

3 Methodology

3.1 Detailed description

In the initial phase of this experiment, the Generative Image-to-Text Transformer (GIT), developed by Microsoft, was selected as the base model for the image captioning task. To adapt the model to the specific characteristics of the dataset, it was fine-tuned using a collection of image-caption pairs from a domain-specific corpus related to Printed Circuit Boards (PCBs).

The fine-tuning process involved optimizing the model to learn the semantic alignment between visual features and textual descriptions. This was achieved by training on a supervised learning objective where the model generated captions for each image, and the outputs were compared against ground-truth labels using standard language generation loss functions.

Following the training phase, the fine-tuned model was evaluated on a separate set of unseen images to assess its generalization capabilities. The objective was to generate contextually accurate and semantically meaningful captions that correctly describe the visual content, including relevant PCB components and potential defects. No specific evaluation metrics were applied to assess the results. This decision was based on the uniformity observed in the model's output: the fine-tuned model consistently generated the same caption across all test cases, regardless of the defects present in the input images. This behavior

indicates a lack of sensitivity to visual variations and limits the usefulness of standard evaluation methods.

3.2 Implementation details

To fine-tune the GIT model on the domain-specific dataset, the implementation leveraged the `Trainer` class provided by the Hugging Face `Transformers` library [32]. This high-level training interface simplifies the process of adapting pre-trained transformer-based models to custom tasks by abstracting away much of the boilerplate code related to optimization, checkpointing, evaluation, and distributed training.

The model was trained in a supervised setting using paired image-caption data. Images were first preprocessed using the corresponding feature extractor associated with the GIT architecture to generate suitable visual inputs for the encoder. Text data was tokenized using the model's tokenizer, ensuring compatibility with the decoder input format.

Key hyperparameters such as learning rate, number of epochs, batch size, and warm-up steps were empirically selected based on preliminary experimentation to balance training stability and convergence speed. Evaluation was conducted periodically on a validation split, using a feature made available by Transformer class itself.

In the second phase of the experiment, a different evaluation strategy was employed using the OpenCLIP model. After fine-tuning the model on the custom PCB dataset, the objective shifted from caption generation to caption selection based on similarity scoring. Specifically, for each image in the test set, the model was provided with multiple candidate captions, of which only one was correct. The model computed a similarity score for each (image, caption) pair, and these scores were subsequently normalized into probabilities via a softmax function across the set of candidate captions.

This setup enabled the evaluation of the model's ability to discriminate between semantically correct and incorrect textual descriptions for a given image. The experiment was repeated under varying conditions, testing the model with sets of 2, 3, and 5 captions per image. The fine-tuned version was evaluated to assess the impact of domain adaptation on selection accuracy. Performance was measured by the model's ability to assign the highest probability to the correct caption across different configurations, providing insights into its semantic alignment capabilities and robustness to distractor captions. The set of candidate captions for each image was constructed by combining the ground-truth caption corresponding to the current image with a selection of previously encountered captions from the dataset. This approach ensured that the incorrect options, or distractors, were realistic and drawn from the same domain, thereby increasing the difficulty and relevance of the evaluation.

For each (image, caption) pair in the candidate set, the model computed a similarity score reflecting the likelihood of semantic correspondence. These scores were subsequently transformed into probabilities using a softmax function across the full set of candidates for a given image. The model's output thus

represented the probability distribution over all candidate captions, with the ideal outcome being a maximal probability assigned to the correct caption. This framework allowed for an assessment of the model's discriminative ability in selecting the correct textual description from a pool of visually plausible alternatives.

4 Evaluation

4.1 Description of datasets

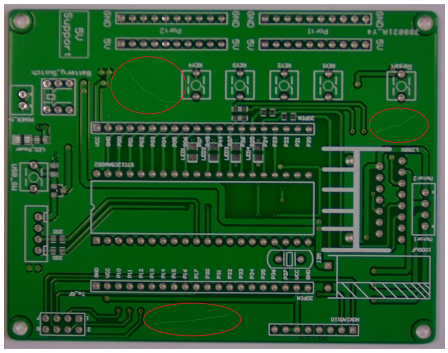
The dataset used for training and evaluation in this study is the PCB_V2 dataset, which is publicly available on Hugging Face Datasets section[3]. This dataset comprises annotated images of printed circuit boards (PCBs), offering both visual content and corresponding textual descriptions suitable for tasks such as image captioning and vision-language model evaluation. All PCB images are related to the entire board, and no cropped or localized regions are provided to assist the model in identifying specific defects.

- **image (image)**: This is a photograph of a printed circuit board, capturing the entire layout in a single frame. It serves as the primary visual input for all model training and evaluation tasks performed in this study.
- **guide (image)**: This feature is an auxiliary black image of the same dimensions as the original board image. It contains several white squares, each precisely placed at the location of a known defect on the board. These white squares function as spatial annotations, offering a visual map of defect locations. Despite its potential utility for supervised tasks such as segmentation or attention guidance, this guide image was not utilized in the scope of the current project.
- **text (string)**: This feature consists of a descriptive sentence or phrase that characterizes the nature of the defects found on the board. It provides a semantic summary intended to align with the visual content, thereby enabling tasks such as image captioning, vision-language alignment, or contrastive learning.

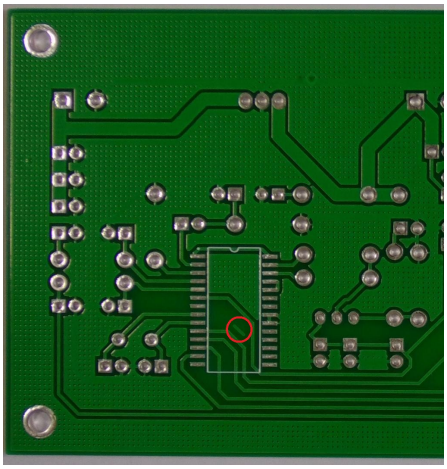
The dataset includes a diverse range of defect types, each of which may occur in varying quantities across different images. These defects represent common issues found in printed circuit boards and appear in multiple configurations, depending on the specific board instance. For each image, red circles are overlaid to visually highlight the presence and precise location of each defect, making them easily distinguishable for both human annotators and machine learning models.

Accompanying each image is a corresponding textual description that details the nature and type of defects present. These captions are aligned with the defects depicted in the visual data, providing a semantic grounding for each listed item. The number of defects annotated per image is not fixed; an individual

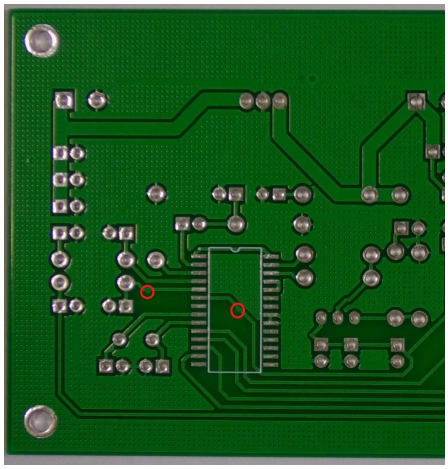
PCB may display as few as one defect or as many as seven, depending on the condition of the board.



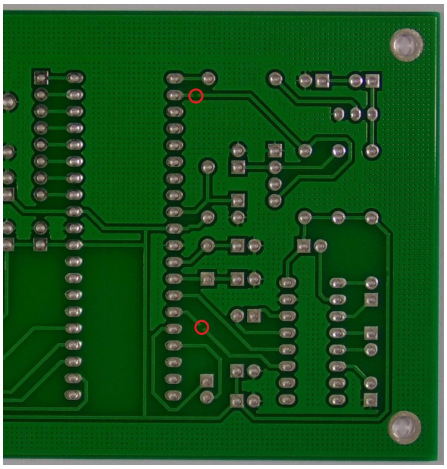
scratch
Plated Circuit Board with 3 scratch defects
Red circles show scratches which may be cosmetic or sever a spur.



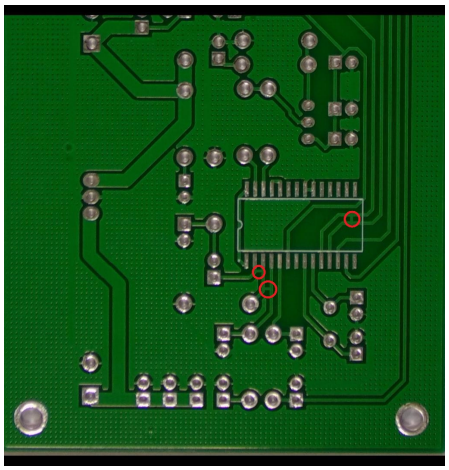
spurious copper
Plated Circuit Board with 1 spurious copper defect
A red circle marks an extra copper spur not needed by the circuit.



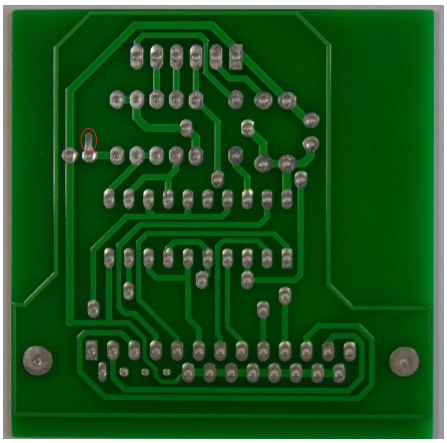
spur
Plated Circuit Board with 2 spur defects
Spur traces highlighted in red are misshaped.



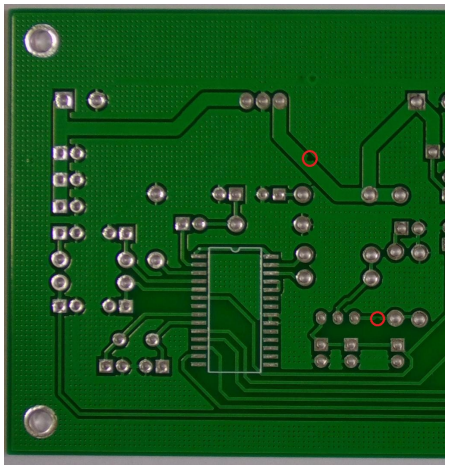
open circuit
Plated Circuit Board with 2 open circuit defects
Missing trace sections can be seen inside red circles.



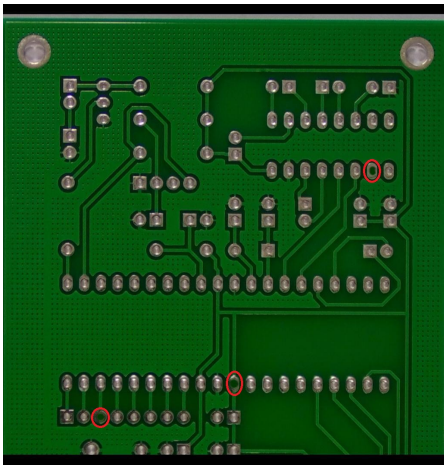
short
Plated Circuit Board with 3 short defects
Some traces are incorrectly connected, creating shorts.



excess solder
Plated Circuit Board with 1 excess solder defect
Red circle shows a pin with excess solder.



mouse bite
Plated Circuit Board with 2 mouse bite defects
Red circles mark irregular trace shapes, possibly cosmetic.



missing hole
Plated Circuit Board with 3 missing holes defects
Missing solder holes could lead to misaligned components.

4.2 Training Methods and Parameters

The training process for the first experiment was conducted using GIT, featuring approximately 177 million parameters. This model operates in an image-to-text framework and was fine-tuned using a batch size of 16 per GPU. The training spanned 50 epochs to ensure sufficient convergence and learning of the mapping

between visual features and descriptive text. Due to memory constraints and computational considerations, it was not feasible to use a larger batch size with GIT, as it would exceed the limits of the available GPU memory.

In contrast, the second experiment employed OpenCLIP, a contrastive vision-language model, which was trained under different conditions. OpenCLIP was fine-tuned for a total of 32 epochs with a significantly larger batch size of 500. This increase in batch size was enabled by the model's architectural efficiency and better scalability, which allowed for more aggressive batching without exceeding hardware limitations.

Both models were trained on the same hardware environment to ensure consistency across experiments. The training system was equipped with two NVIDIA RTX 3090 GPUs, which provided the necessary computational resources for processing large-scale image-text data and executing multiple training steps in parallel.

4.3 Presentation of the Obtained Results

The outcomes derived from both experiments have proven to be underwhelming, falling short of expectations in terms of practical utility and accuracy. In the first experiment, although the model achieved numerically respectable evaluation metrics — a word error rate (WER) of 0.387 and an evaluation loss of 0.010 — these results did not translate into meaningful visual understanding or defect detection. Specifically, the model consistently failed to identify and describe the actual defects present in the input images.

This discrepancy between metric performance and qualitative output highlights a fundamental limitation of the model's comprehension capabilities in this domain. Despite generating text that may appear fluent or syntactically correct, the model does not demonstrate an ability to localize or semantically interpret the defective regions of the printed circuit boards (PCBs). Consequently, its predictions lack relevance and accuracy when assessed against the ground-truth annotations provided in the dataset.

For the second experiment, the evaluation was conducted using a different scoring methodology. Specifically, the model was provided with a set of candidate captions for each image, and its task was to assign a probability to each one. These captions included the ground-truth description as well as distractors drawn from other samples. The caption with the highest assigned probability was then compared against the correct one. The overall accuracy score was computed as the proportion of correctly selected captions out of the total number of evaluated images.

Across all tested caption set sizes, the model consistently performed worse than a random baseline. The detailed results are as follows:

- With 2 candidate captions, the model achieved an accuracy of 0.441, whereas a random selection would be expected to yield 0.500.
- With 3 candidate captions, the model achieved 0.197, in contrast to the random baseline of 0.333.

- With 5 candidate captions, the result dropped further to 0.118, below the 0.200 expected by chance.

These findings suggest that, regardless of the number of caption choices, the model consistently fails to correctly associate the images with their appropriate textual descriptions. In conclusion, the performance observed in both experiments indicates that the models are not capable of recognizing or distinguishing any meaningful visual differences between the images, rendering them ineffective for the defect identification task at hand.

5 Conclusion

This experiment set out to explore the feasibility of automating quality control processes through the use of image captioning techniques. By leveraging state-of-the-art vision-language models, the aim was to generate descriptive textual outputs that accurately reflect the presence and nature of defects in printed circuit board (PCB) images. Two prominent approaches were investigated: direct image-to-text generation using a pre-trained captioning model (GIT), and contrastive learning with text-image pairing via OpenCLIP.

Despite the use of well-established models and a carefully constructed dataset, the outcomes from both experimental setups were ultimately unsatisfactory. The captioning model, although achieving favorable loss metrics and word error rate scores, consistently failed to produce descriptions that corresponded to the actual defects in the images. Similarly, the contrastive model was unable to distinguish correct captions from incorrect ones with any meaningful accuracy, performing consistently below random chance across all tested configurations.

These results highlight the current limitations of applying generic image-text models to highly specialized industrial tasks such as PCB defect identification. The failure to generalize or fine-tune effectively in this context suggests the need for more targeted datasets, task-specific model architectures, or hybrid approaches combining vision-language understanding with structured domain knowledge.

In summary, while the goal of automating quality control through image captioning remains promising in theory, the findings of this experiment underline the gap between current general-purpose models and the demands of real-world manufacturing applications.

5.1 Discussion

The results obtained in both experiments suggest that achieving high precision in the automatic captioning of PCB defect images remains a significant challenge with current methods. Specifically, the first experiment, which utilized a generative image-to-text model, demonstrated a fundamental limitation: the model consistently produced the same caption for all input images, regardless of the content or presence of defects. This uniformity in output implies that the

model failed to learn any meaningful correlation between visual features and the associated textual descriptions during fine-tuning.

In the second experiment, which adopted a contrastive learning approach, expectations were that the model would at least be able to distinguish between correct and incorrect captions based on learned embeddings. However, the outcomes were equally discouraging. The model frequently assigned the highest probability to incorrect captions, and the overall accuracy across multiple-choice evaluations was consistently below the baseline expected from random guessing. For example, with two caption options, the accuracy was 0.441 (below the 0.500 random benchmark), and with five options, the accuracy dropped as low as 0.118, significantly underperforming compared to the 0.200 random benchmark.

These findings indicate not only a failure in learning discriminative representations but also suggest that neither of the models adapted effectively to the specific nature of the dataset. The uniformity of visual scenes, lack of high-resolution defect annotation (e.g., no cropped defect-specific sub-images), and the fine-grained nature of the required distinctions may have all contributed to the ineffectiveness of both generative and contrastive approaches. Additionally, the relatively small dataset size compared to the scale these models are designed for could have limited the learning signal available during fine-tuning or training.

Overall, the experiments highlight a substantial gap between the capabilities of current vision-language models and the demands of domain-specific tasks such as PCB defect analysis. Addressing this gap may require more customized datasets, architectures with stronger inductive biases for localized defect detection, or multimodal pipelines that explicitly incorporate object detection or segmentation alongside captioning.

5.2 Takeaways

One positive outcome of this study is that the training processes for both experiments were executed correctly and completed without technical issues. The models were able to converge, losses decreased as expected, and no abnormal behavior was observed during the optimization steps. This confirms that the training pipelines—data preprocessing, model setup, and hardware configuration—were all correctly implemented and operational.

However, successful training alone does not guarantee a useful or performant model. Despite the correct training procedure, the models failed to learn meaningful patterns that correlate image features with the appropriate textual descriptions. This emphasizes a crucial insight: a functioning training process is a necessary but not sufficient condition for model effectiveness. The actual utility of the model depends on how well it captures the underlying data distributions and performs the intended task in a reliable and discriminative manner.

The outcomes suggest that more targeted model architectures, enhanced datasets, or alternative task formulations may be required to achieve satisfactory performance in complex vision-language problems like defect captioning.

5.3 What Can Be Done in the Future to Improve the Results

Several directions can be explored to potentially enhance the performance of the models in future experiments. One of the primary challenges identified in this study is the high visual similarity between different images, even when distinct defects are present. This similarity likely confuses the models, making it difficult to associate visual variations with their corresponding textual descriptions.

A potential improvement could involve preprocessing the dataset to isolate or highlight only the defective regions in each image. By removing irrelevant background information and focusing the model's attention on the affected areas, the learning process may become more effective and lead to better discrimination between defect types.

Another promising avenue is to significantly expand the training dataset. Increasing the number and diversity of image-text pairs could provide the model with a broader variety of visual patterns and linguistic structures, helping it to learn finer distinctions between similar-looking images. A larger dataset could also help mitigate overfitting and improve generalization across unseen samples.

Additionally, incorporating defect-localization guidance (such as the guide feature already present in the dataset) into the training process could further enhance model performance by explicitly signaling where the model should attend within the image.

Ultimately, combining more focused visual input, larger-scale data, and possibly more specialized architectures or multi-stage training techniques may lead to substantial improvements in performance on this complex task.

6 Code

7 Code and Experimental Setup

This section provides the complete codebase employed for conducting the experiments described in this study, along with relevant technical specifications and configuration details.

- Python: 3.12.3
- Jupyter-Notebook
  - IPython : 9.2.0
  - ipykernel: 6.29.5
  - ipywidgets: 8.1.7
  - jupyter_client: 8.6.3
  - jupyter_core: 5.7.2
  - jupyter_server: 2.15.0

  - jupyterlab: 4.4.2
  - nbclient: 0.10.2
  - nbconvert: 7.16.6
  - nbformat: 5.10.4
  - notebook: 7.4.2
  - qtconsole: not installed
  - traitlets: 5.14.3
- Python packages:
  - numpy: 2.2.5
  - transformers: 4.51.3
  - evaluate: 0.4.3
  - open_clip_torch: 2.32.0
  - torch: 2.7.1+cu128
  - torchaudio: 2.7.1+cu128
  - torchvision: 0.22.1+cu128
  - requests: 2.32.3
  - accelerate: 1.6.0
  - datasets: 3.6.0
  - pandas: 2.2.3
  - pyarrow: 20.0.0
- OS: Linux testa 6.8.0-59-generic #61-Ubuntu SMP PREEMPT_DYNAMIC Fri Apr 11 23:16:11 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
- CPU: AMD EPYC 7452 32-Core Processor (4 cores available in the system)
- RAM: 16GiB
- GPU: NVIDIA GeForce RTX 3090 + NVIDIA GeForce RTX 3090
- Swap: 4GiB
- Storage: 100GiB

7.1 GIT

For the GIT model, the entire workflow—including the download of the pre-trained weights, the training procedure, and the generation of text captions—has been implemented using Python.

```
from textwrap import wrap
import numpy as np
from transformers import AutoProcessor
from transformers import AutoModelForCausalLM
from evaluate import load
import torch
from transformers import TrainingArguments, Trainer
from PIL import Image
import requests
from accelerate.test_utils.testing import get_backend
import datasets
from datasets import load_dataset
import pandas as pd
import pyarrow as pa



token="..."
ds = load_dataset("chengjr/PCB_V2")

train_ds = ds["train"]
test_ds = ds["validation"].train_test_split(test_size
    =0.25) #reduce the size to avoid overflow error during
     training
test_ds = test_ds["test"]

checkpoint = "microsoft/git-base"
processor = AutoProcessor.from_pretrained(checkpoint,
    token=token, use_fast=False)

def transforms(example_batch):
    images = [x for x in example_batch["image"]]
    captions = [x for x in example_batch["text"]]
    inputs = processor(images=images, text=captions,
        padding="max_length")
    inputs.update({"labels": inputs["input_ids"]})
    return inputs

train_ds.set_transform(transforms)
test_ds.set_transform(transforms)
```

```
checkpoint = "microsoft/git-base"
processor = AutoProcessor.from_pretrained(checkpoint,
    token=token, use_fast=False)

def transforms(example_batch):
    images = [x for x in example_batch["image"]]
    captions = [x for x in example_batch["text"]]
    inputs = processor(images=images, text=captions,
        padding="max_length")
    inputs.update({"labels": inputs["input_ids"]})
    return inputs

train_ds.set_transform(transforms)
test_ds.set_transform(transforms)



device = torch.device("cuda" if torch.cuda.is_available()
    else "cpu")
wer = load("wer")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predicted = logits.argmax(-1)
    decoded_labels = processor.batch_decode(labels,
        skip_special_tokens=True)
    decoded_predictions = processor.batch_decode(
        predicted, skip_special_tokens=True)
    wer_score = wer.compute(predictions=
        decoded_predictions, references=decoded_labels)
    return {"wer_score": wer_score}

def compute_diff(predictions=None, references=None,
    concatenate_texts=False):
    if concatenate_texts:
        return compute_measures(references, predictions)[
            "wer"]
    else:
        incorrect = 0
        total = 0
        for prediction, reference in zip(predictions,
            references):
            arrp=prediction.split("-")
            arrr=reference.split("-")
            for i in range(len(arrp)):
```

```
                total += 1
                try:
                    incorrect += (arrp[i]!=arrr[i])
                except:
                    incorrect += 1
        return incorrect/total

def precision_metrics(eval_pred):
    logits, labels = eval_pred
    predicted = logits.argmax(-1)
    decoded_labels = processor.batch_decode(labels,
        skip_special_tokens=True)
    decoded_predictions = processor.batch_decode(
        predicted, skip_special_tokens=True)
    computedDiff = compute_diff(predictions=
        decoded_predictions, references=decoded_labels)
    return {"wer_score": computedDiff}



#Train
import torch.nn as nn
from transformers import AutoProcessor
from trl import SFTTrainer

model = AutoModelForCausalLM.from_pretrained(checkpoint,
    token=token).to(device)
model_name = checkpoint.split("/")[1]

training_args = TrainingArguments(
    output_dir="git-base-pcb",
    learning_rate=5e-5,
    num_train_epochs=50,
    fp16=True,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    gradient_accumulation_steps=2,
    save_total_limit=50,
    eval_strategy="steps",
    eval_steps=50,
    save_strategy="steps",
    #save_strategy="epoch",
    save_steps=50,
    logging_steps=50,
    remove_unused_columns=False,
    push_to_hub=False,
```

```
    label_names=["labels"],
    load_best_model_at_end=True,
    report_to="none",
    ddp_find_unused_parameters=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=test_ds,
    compute_metrics=compute_metrics,
)

trainer.train()



for num in range(len(ds['test'])):
    defect = (ds['test'][num]['text'].split("with-")[1].
        split("-defects")[0])
    image = (ds['test'][num]['image'])
    model = AutoModelForCausalLM.from_pretrained(f"{
        model_name}-pcb/checkpoint-950/", device_map="cuda
        ")
    #processor = AutoProcessor.from_pretrained(checkpoint
        , token=token, use_fast=False)
    device, _, _ = get_backend()
    inputs = processor(images=image, return_tensors="pt")
        .to(device)
    pixel_values = inputs.pixel_values
    generated_ids = model.generate(pixel_values=
        pixel_values, max_length=50)
    generated_caption = processor.batch_decode(
        generated_ids, skip_special_tokens=True)[0]
    print(generated_caption)
```

7.2 OpenCLIP

For the OpenCLIP model, the dataset was downloaded using Python scripts, while the training phase was executed through a single Bash command. The generation of textual outputs was subsequently performed in Python.

```
import datasets
from datasets import load_dataset
```

```
token="..."
ds = load_dataset("chengjr/PCB_V2")
train_ds = ds["train"]
test_ds = ds["validation"].train_test_split(test_size
    =0.25) #same size as previous experiment
test_ds = test_ds["test"]



#Let's create the csv file according to specifications
!mkdir -p /home/robalb/PCB_V2/training/
i=0
f = open('/home/robalb/train.csv', 'a')
f.write('filepath,caption\n')
for entry in train_ds:
    entry['image'].save('/home/robalb/PCB_V2/training/'+
        str(i)+'.jpg')
    f.write('/home/robalb/PCB_V2/training/'+str(i)+'.jpg
        ,"'+entry['text']+'"\n')
    i+=1
f.close()
```

Once the CSV file has been generated using Python, the model must be trained on the corresponding CSV file and image dataset. This training procedure is carried out via the terminal using Bash.

```
torchrun \
--nproc_per_node 2 -m open_clip_train.main \
--batch-size 500 \
--precision amp \
--workers 4 \
--report-to none \
--save-frequency 1 \
--dataset-type csv \
--csv-separator="," \
--train-data /home/robalb/train.csv \
--csv-img-key filepath \
--csv-caption-key caption \
--warmup 1000 \
--lr=5e-6 \
--wd=0.1 \
--epochs=32 \
--model ViT-B-32 \
--pretrained 'laion2b_s34b_b79k'
```

Once training is complete, the process returns to Python to evaluate the model. In this version, the code tests the generated caption against the two preceding ones from the dataset. As a result, the expected accuracy for a random guess is approximately 33%.

```
import torch
from PIL import Image
import open_clip

model, _, preprocess = open_clip.
    create_model_and_transforms('ViT-B-32', pretrained='/
    home/robalb/logs/2025_06_28-18_10_12-model_ViT-B-32-
    lr_5e-06-b_500-j_4-p_amp/checkpoints/epoch_32.pt')
tokenizer = open_clip.get_tokenizer('ViT-B-32')
rights=0

for entry in range(len(ds["validation"])):

    image = preprocess(ds["validation"][entry]['image']).
        unsqueeze(0)
    text = tokenizer([ds["validation"][entry]['text'], ds
        ["validation"][entry-1]['text'], ds["validation"][
        entry-2]['text']])

    with torch.no_grad(), torch.amp.autocast('cuda'):
        image_features = model.encode_image(image)
        text_features = model.encode_text(text)
        image_features /= image_features.norm(dim=-1,
            keepdim=True)
        text_features /= text_features.norm(dim=-1,
            keepdim=True)

        text_probs = (100.0 * image_features @
            text_features.T).softmax(dim=-1)

        if (text_probs[0].sort().indices[-1]==0):
            rights+=1
print ("Ratio of correct answers:", rights/len(ds["
    validation"]))
```

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] chengjr. `https://huggingface.co/datasets/chengjr/PCB_V2`. Last access: July, 24, 2025.

[4] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2818–2829, 2023.

[5] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023.

[6] Erik CW De Jong, Braham JA Ferreira, and Pavol Bauer. Toward the next level of pcb usage in power electronic converters. *IEEE Transactions on Power Electronics*, 23(6):3151–3163, 2008.

[7] Karan Desai and Justin Johnson. Virtex: Learning visual representations from textual annotations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11162–11173, 2021.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[9] Mingyu Ding, Bin Xiao, Noel Codella, Ping Luo, Jingdong Wang, and Lu Yuan. Davit: Dual attention vision transformers. In *European conference on computer vision*, pages 74–92. Springer, 2022.

[10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016.

[14] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, and Mu Li. Bag of tricks for image classification with convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 558–567, 2019.

[15] Xiaowei Hu, Zhe Gan, Jianfeng Wang, Zhengyuan Yang, Zicheng Liu, Yumao Lu, and Lijuan Wang. Scaling up vision-language pre-training for image captioning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17980–17989, 2022.

[16] WC Leong, Mohd Zulkifly Abdullah, and CY Khor. Application of flexible printed circuit board (fpcb) in personal computer motherboards: focusing on mechanical performance. *Microelectronics Reliability*, 52(4):744–756, 2012.

[17] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.

[18] Madhav Moganti, Fikret Ercal, Cihan H Dagli, and Shou Tsunekawa. Automatic pcb inspection algorithms: a survey. *Computer vision and image understanding*, 63(2):287–313, 1996.

[19] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.

[20] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

[21] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[22] V Udaya Sankar, Gayathri Lakshmi, and Y Siva Sankar. A review of various defects in pcb. *Journal of Electronic Testing*, 38(5):481–491, 2022.

[23] Sara Sarto, Marcella Cornia, and Rita Cucchiara. Image captioning evaluation in the age of multimodal llms: Challenges and future perspectives. *arXiv preprint arXiv:2503.14604*, 2025.

[24] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in neural information processing systems*, 35:25278–25294, 2022.

[25] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. Laion-400m: Open dataset of clip-filtered 400 million image-text pairs. *arXiv preprint arXiv:2111.02114*, 2021.

[26] Hamed Shamkhalichenar, Collin J Bueche, and Jin-Woo Choi. Printed circuit board (pcb) technology for electrochemical sensors and sensing platforms. *Biosensors*, 10(11):159, 2020.

[27] Kihyuk Sohn. Improved deep metric learning with multi-class n-pair loss objective. *Advances in neural information processing systems*, 29, 2016.

[28] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[29] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. In *European conference on computer vision*, pages 776–794. Springer, 2020.

[30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[31] Jianfeng Wang, Zhengyuan Yang, Xiaowei Hu, Linjie Li, Kevin Lin, Zhe Gan, Zicheng Liu, Ce Liu, and Lijuan Wang. Git: A generative image-to-text transformer for vision and language. *arXiv preprint arXiv:2205.14100*, 2022.

[32] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.

[33] Bin Xiao, Haiping Wu, Weijian Xu, Xiyang Dai, Houdong Hu, Yumao Lu, Michael Zeng, Ce Liu, and Lu Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4818–4829, 2024.

[34] Lu Yuan, Dongdong Chen, Yi-Ling Chen, Noel Codella, Xiyang Dai, Jianfeng Gao, Houdong Hu, Xuedong Huang, Boxin Li, Chunyuan Li, et al. Florence: A new foundation model for computer vision. *arXiv preprint arXiv:2111.11432*, 2021.

[35] Pengchuan Zhang, Xiujun Li, Xiaowei Hu, Jianwei Yang, Lei Zhang, Lijuan Wang, Yejin Choi, and Jianfeng Gao. Vinvl: Revisiting visual representations in vision-language models, 2021.

[36] Richard Zhang. Making convolutional networks shift-invariant again. In *International conference on machine learning*, pages 7324–7334. PMLR, 2019.

[37] Yuhao Zhang, Hang Jiang, Yasuhide Miura, Christopher D Manning, and Curtis P Langlotz. Contrastive learning of medical visual representations from paired images and text. In *Machine learning for healthcare conference*, pages 2–25. PMLR, 2022.